Experiment No. 09

DESIGN OF TEST CASES

Doctor Appointment Booking System — Aurelia Care

Student details	Information
Name	Ronit Rajput
Roll No.	E052
Program	Computer Science and Business Systems
Division / Batch	E / 2
Date of Experiment	26 October 2026
Date of Submission	_____
Grade	_____

PART A

A.1 Aim

Design test cases for any two functionalities of the project: user login and appointment booking.

A.2 Prerequisites

Unit testing, black-box testing, white-box testing, and knowledge of the project requirements.

A.3 Outcome

Document and execute test cases with expected results, actual results, pass/fail status, and supporting evidence.

A.4 Theory

Software testing compares specified requirements with observed behaviour to detect defects. A test case defines preconditions, input data, execution steps, and expected results; execution supplies the actual result and verdict.

Black-box testing examines externally observable behaviour. White-box testing uses internal logic to exercise statements, decisions, and error paths. Unit tests isolate a small function; API tests using the database are integration tests.

Testing activities include setting objectives, identifying scenarios, preparing data, writing and executing cases, comparing results, reporting defects, and retesting. Key documents are the test plan, test case specification, incident report, and test summary.

A.4 Task (as numbered in the manual)

Write appropriate test cases for the project, considering any two functionalities.

PART B

B.1 Tasks Given in Part A

1. Scope and test plan

Aurelia Care supports patient authentication, doctor search, appointment booking, schedules, and administration. This experiment selects only login and appointment booking. Supporting checks inspect authentication, payment failure, time boundaries, concurrency, and transaction rollback.

Item	Configuration
Source version	b9663ffdcb771e8cf48beff67546490351794bcb
Application	React frontend; FastAPI backend; SQLite database
Execution environment	Linux cloud workspace; Python 3.12; Chromium browser
Focused tools	pytest, FastAPI TestClient, coverage.py 7.17.0, Playwright
Data isolation	Temporary SQLite fixtures for API tests; separate lab database for browser evidence
Priority / verdict	H = High; M = Medium. Pass requires all assertions in the case to succeed.

2. Test data

Field	Fictional laboratory value
Patient	Aarav Shah — patient@example.com
Second patient	Diya Patel — diya@example.com
Test password	TestingPass123! (isolated test fixtures only)
Incorrect password	WrongPass123!
Unknown email	unknown@example.com
Doctor / consultation fee	Dr. Jhatka / INR 1,200
Appointment slots	Generated future slots; actual identifiers selected from the availability API
Time-boundary fixture	15 January 2030, 10:00 IST; clock frozen for boundary tests
Payment	Simulation: success or failure; no real payment processing

The password above is a test fixture, not an installation login credential. Browser tests use separately generated local credentials, which are not included in this report. Existing user data and the main database were not used for lab mutations.

3. Test design approach

- Equivalence partitioning: valid, incorrect, missing, malformed, and inactive login inputs.
- State transitions: logged out → authenticated → logged out; available slot → booked slot.
- Boundary analysis: appointment starts one minute before, exactly at, and one minute after a frozen current time.
- White-box analysis: login rejection and success, availability guards, payment branch, password verifier, and rollback after writes.

The next two tables retain all ten fields from the manual’s test case specification template. They use landscape pages for readability. API observations are distinguished from browser observations.

4. Test Cases: User Login

Method: black-box API integration testing. Each case has an isolated fixture; LG-02 also has browser evidence.

Test case ID	Test case	Priority	Preconditions	Input test data	Steps to be executed	Expected results	Actual results	Pass / fail	Comments
LG-01	Valid patient login	H	Active patient; logged out	Registered email; correct password	POST login; GET account	200; patient identity; usable session	200; patient role; /me 200; HttpOnly cookie	Pass	API authentication
LG-02	Incorrect password	H	Active patient; logged out	Registered email; WrongPass123!	POST login	401; login rejected	401 returned	Pass	Browser error also verified
LG-03	Unknown account	H	Email not registered	unknown@example.com; test password	POST login	401; login rejected	401 returned	Pass	Negative partition
LG-04	Empty required fields	M	Logged out	Empty email and password	POST login with empty strings	422; validation fails	422 returned	Pass	API input validation
LG-05	Malformed email	M	Logged out	aarav.test@; test password	POST login	422; invalid email	422 returned	Pass	API input validation
LG-06	Inactive account	H	Patient deactivated in fixture	Correct email and password	Deactivate fixture; POST login	401; login rejected	401 returned	Pass	Account-status decision
LG-07	Account roles and access	H	Active patient, doctor, admin	Valid credentials per role	Login each role; patient requests admin users	Correct role; patient admin access 403	All 3 roles matched; restricted request 403	Pass	API roles; no claim of all UI routes
LG-08	Session revoked on logout	H	Patient logged in	Session cookie	Logout; replay saved cookie to /me	Logout 200; replay 401	200 then 401	Pass	Session lifecycle

5. Test Cases: Appointment Booking

Method: black-box API integration tests with database assertions where stated. Payment is simulated.

Test case ID	Test case	Priority	Preconditions	Input test data	Steps to be executed	Expected results	Actual results	Pass / fail	Comments
BK-01	Successful booking	H	Patient authenticated; future slot free	Future slot; payment success	POST booking; inspect database	201; one booking; paid; slot booked	201; confirmed; simulated_paid; INR 1,200; +1 booking	Pass	API and browser confirmation
BK-02	Unauthenticated request	H	Logged-out client; valid slot	Future slot ID	POST booking without session	401; no database changes	401; snapshot unchanged	Pass	Authentication
BK-03	Occupied slot	H	First patient already booked slot	Second patient; same slot	Book slot again; compare records	409; no duplicate changes	409; snapshot unchanged	Pass	Availability guard
BK-04	Past appointment	H	Logged-in patient; past slot fixture	Date 2000-01-01	POST booking; compare records	422; no booking	422; snapshot unchanged	Pass	Past-time guard
BK-05	Payment declined	H	Available future slot	payment_result=failure	POST booking; inspect records	402; no saved booking; slot free	402; all four table snapshots unchanged	Pass	Browser failure message verified
BK-06	Retry failed payment	H	Future slot initially free	Failure then success; same slot	Fail payment; retry successfully	402 then 201; exactly one new booking	402 then 201; appointment count +1	Pass	Browser retry verified
BK-07	Concurrent requests	H	One available slot; valid patient session	8 simultaneous requests	Submit with 8 workers; query records	One 201; seven 409; one confirmed booking	Exactly one 201 and seven 409; one confirmed row	Pass	Same patient session; 8 requests
BK-08	Booking persistence	H	Successful booking exists	Saved appointment ID	Logout; reinitialize schema; login; GET booking	Saved record remains identical	200; full response equals saved response	Pass	New authenticated client

6. White-Box and Supporting Tests

Source inspection identified the login rejection condition, password verification, slot guards, payment branch, and transaction boundaries. The following seven additional pytest cases exercise these paths. They complement the sixteen functional cases.

ID	Internal path / technique	Observed result	Verdict
WB-01a	Slot starts one minute before frozen time	422; past slot rejected	Pass
WB-01b	Slot starts exactly at frozen time	422; equality boundary rejected	Pass
WB-01c	Slot starts one minute after frozen time	201; future booking accepted	Pass
WB-02	Missing slot; doctor role calls patient booking API	404 for missing slot; 403 for doctor	Pass
WB-03	Slot belongs to inactive doctor	404; slot rejected	Pass
WB-04	Fault injected in db.notify after appointment, slot and payment writes	500 returned; appointments, payments, slots and notifications exactly restored	Pass
WB-05	Password verifier unit test: match, mismatch, malformed encoding	True, False, False respectively	Pass

6.1 Measured source coverage

The focused 23-case run was instrumented with coverage.py branch tracing. The figures below apply only to the named functions, not the whole application.

Function	Statements covered	Branch outcomes covered
main.login	6 / 6 (100%)	2 / 2 (100%)
appointments.available	8 / 8 (100%)	6 / 6 (100%)
appointments.book	12 / 12 (100%)	4 / 4 (100%)
security.verify	6 / 6 (100%)	0 measured branches (not applicable)

Coverage measures executed statements and branch arcs. It does not establish complete path coverage, condition/MC/DC coverage, or absence of defects. Other functions, such as cancellation and Excel import, are outside this focused coverage claim.

6.2 Test summary

Run	Result
Focused Experiment 9 suite	23 passed; 0 failed; 0 skipped — 27.56 seconds
Existing regression suite	21 passed; 0 failed; 0 skipped — 17.59 seconds
Fresh browser checks	4 scenarios passed: rejected login, successful login, failed payment, successful retry / confirmation
Warnings	One upstream Starlette / AnyIO deprecation warning in each API run; no test assertions disabled

7. Browser Evidence

Screenshots were captured from the running application using a separate fictional lab database. These are observed browser states, not mockups.

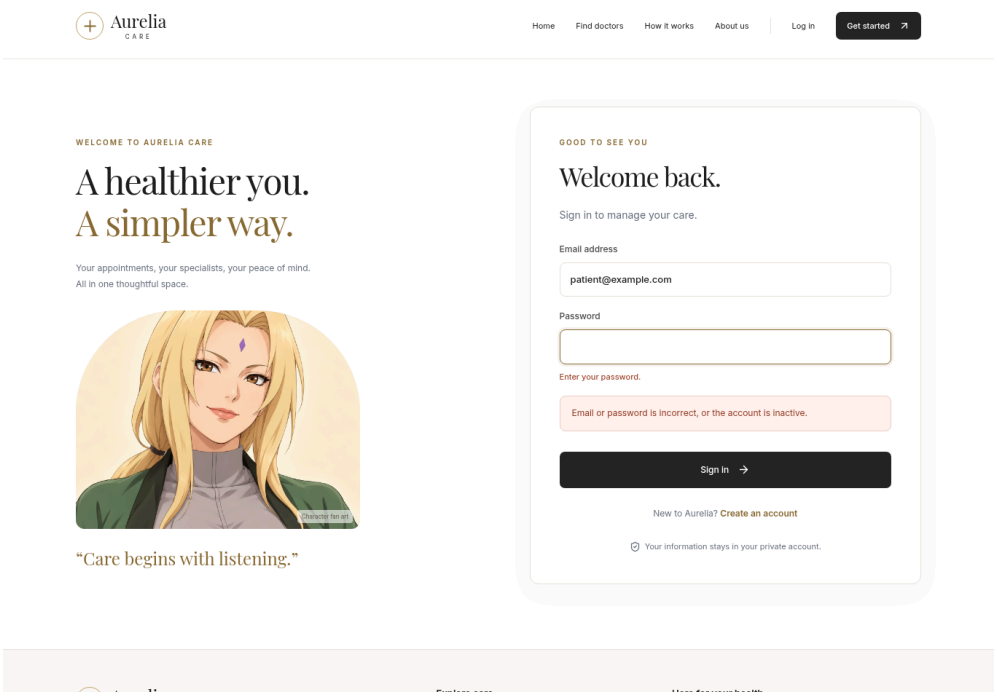


Figure 1. Incorrect credentials rejected with the application's generic login error (LG-02).

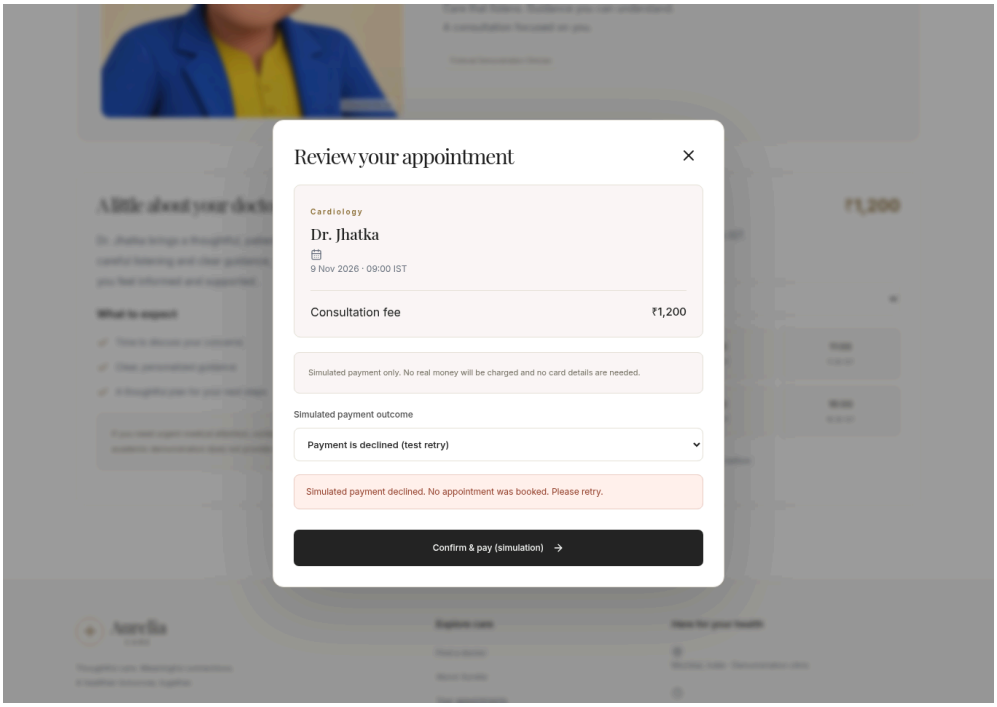


Figure 2. Simulated payment declined; the dialog reports that no appointment was booked (BK-05).

7. Browser Evidence (continued)

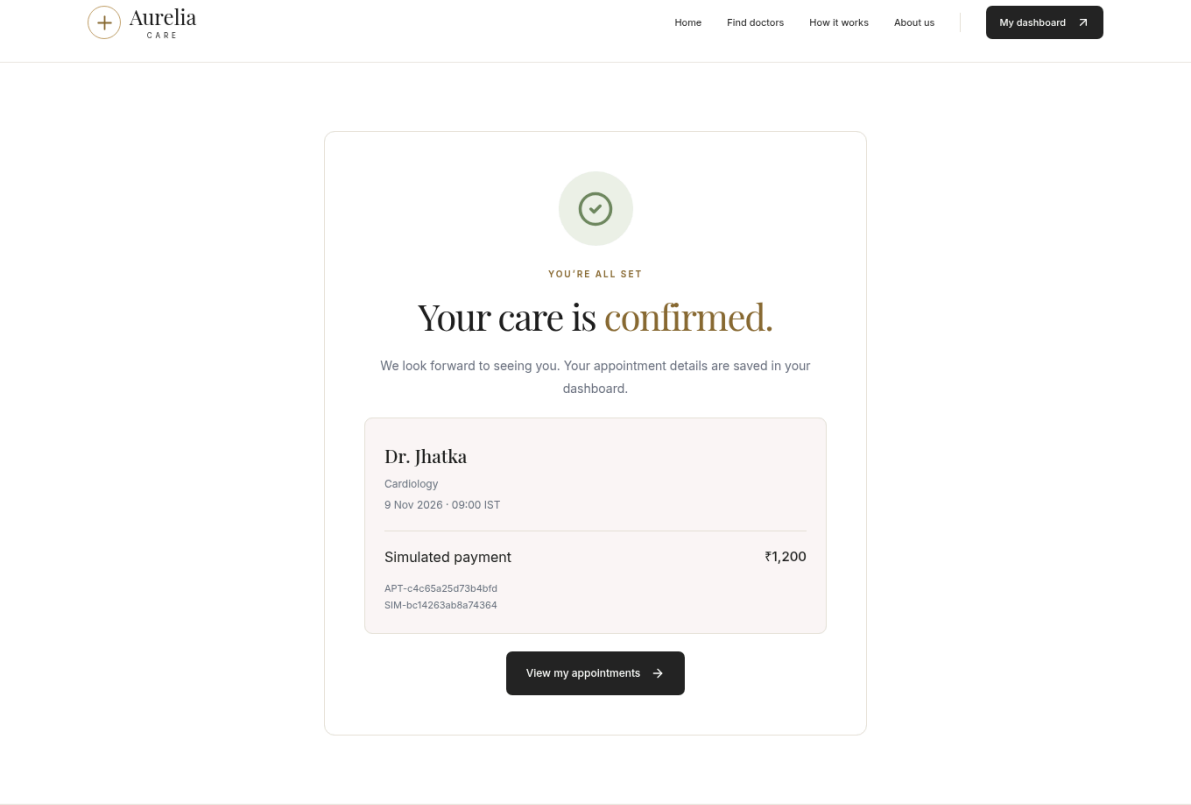


Figure 3. Successful retry creates a confirmed appointment with simulated payment (BK-01 / BK-06).

7.1 Execution evidence

Machine-readable JUnit results, the focused run log, coverage JSON, test source, and browser script are included in the accompanying evidence archive. The exact API outcomes used in the case tables are asserted by those tests.

Command / evidence	Purpose
pytest tests -q	Existing 21-test regression suite
coverage run --branch ... pytest test_experiment9.py -v	Focused 23-test run with source coverage
coverage json	Function-level statement and branch metrics
node capture.mjs	Four browser checks and three screenshots

An initial browser attempt on the lab's alternate port was rejected by the existing Origin safeguard. The lab server was restarted with that exact local origin in ALLOWED_ORIGINS; the completed browser run then passed. Application source code was unchanged. The initial pre-permission API process was stopped and rerun with supported local networking.

B.2 Observations and Learning

- Valid credentials produced a usable HttpOnly-cookie session. Wrong, unknown, and inactive-account login requests were rejected. Empty and malformed fields returned validation errors.
- Logging out invalidated the saved session: replaying the previous cookie returned 401. Role checks denied a patient access to administrator user management.
- Successful booking saved a confirmed appointment, matching simulated payment, and a booked slot. A new authenticated client retrieved the same record.
- Eight concurrent requests for one slot produced one success and seven conflicts, with only one confirmed database row.
- A simulated payment failure returned 402 without changing appointments, payments, availability, or notifications. Retrying successfully created only one new booking.
- The frozen-clock tests demonstrated the exact time rule: a slot at or before the current time is rejected; a later slot can be booked.
- A deliberate exception after database writes caused transaction rollback. No partial booking, payment, notification, or slot-state change remained.
- Black-box observations and white-box source tracing provide complementary evidence. Complete branch coverage of selected functions does not imply complete coverage of the application.

B.3 Conclusion

The aim was achieved by designing and executing test cases for login and appointment booking in the Doctor Appointment Booking System. All sixteen functional cases and seven supporting white-box/boundary cases passed. The twenty-one existing regression tests also passed, and four fresh browser scenarios supplied visual evidence.

The results support the tested authentication, booking, failure-handling, concurrency, and persistence behaviours for the identified source version in the Linux lab environment. They do not establish production performance, native Mac compatibility, or exhaustive correctness.

B.4 Questions of Curiosity

1. What are black-box and white-box testing? Differentiate between them.

Black-box testing checks software through inputs and externally visible outputs, using requirements rather than internal code to select cases. White-box testing uses source structure and logic to select execution paths and measure structural coverage.

Basis	Black-box testing	White-box testing
Focus	External behaviour	Internal implementation
Test design basis	Requirements and specifications	Source code and control flow
Code knowledge	Not required	Required
Techniques	Equivalence partitions, boundaries, decision tables, state transitions	Statement, branch and condition coverage; path analysis
Project example	Occupied slot must be rejected	Exercise availability guards and payment branches; inspect rollback
Limitation	May miss internal paths	May miss a requirement omitted from the code

Both approaches are useful at multiple testing levels. Black-box testing is not restricted to manual tests, and white-box testing is not synonymous with unit testing.

2. What are stubs and drivers? Differentiate between them.

A stub temporarily replaces a dependency called by the component under test and returns controlled responses. A driver temporarily replaces a caller: it invokes the target component, supplies inputs, and checks outputs.

Basis	Stub	Driver
Replaces	Called dependency	Calling component
Position	Below component under test	Above component under test
Typical integration approach	Top-down	Bottom-up
Example	Fake payment dependency returning success or failure	Harness invoking the booking service with fixture inputs

Relationship: Driver → Component under test → Stub. These aids simplify testing but may not reproduce every real integration behaviour. In this experiment, the injected notification failure is a fault-injection replacement; the application’s payment-outcome selector is an existing simulation feature.

B.4 Questions of Curiosity (continued)

3. Explain the advantages and disadvantages of top-down compared with bottom-up integration.

Top-down integration begins with higher-level control modules and progressively integrates lower-level dependencies. Stubs stand in for dependencies that are not yet available. In this project, the booking API could first be tested with controlled slot, payment, and persistence dependencies.

Advantages of top-down integration

- Major workflows and orchestration can be checked early.
- An early system skeleton supports demonstrations and requirements feedback.
- Interface and control-flow problems between higher-level modules can be identified before all lower-level services are complete.

Disadvantages of top-down integration

- Many realistic stubs may be needed and must be maintained.
- Detailed database and utility behaviour is verified later.
- Simplified dependency responses may hide faults until the real components are integrated.

Bottom-up integration begins with low-level services and combines them into larger components. Drivers simulate unavailable callers. In this project, password verification, database constraints, and slot reservation could be checked before integrating the API and interface.

Advantages of bottom-up integration

- Foundational data operations and utilities are verified early.
- Failures in small components are generally easier to isolate.
- Higher-level integration can build on already-tested lower-level services.

Disadvantages of bottom-up integration

- Drivers must be written and maintained.
- Complete user workflows and demonstrations become available later.
- High-level coordination and navigation faults may remain undiscovered until later integration.

Aspect	Top-down	Bottom-up
Starting point	High-level control modules	Low-level services
Main aid	Stubs	Drivers
Early visibility	Main workflows	Foundational operations
Later integration	Detailed dependencies	Overall orchestration

A combined or sandwich approach is suitable for Aurelia Care: test authentication and database operations independently, integrate the API and its dependencies, then verify complete browser workflows.